Universität Konstanz

Introduction to Computing for the Social Sciences

Session 5 – Algorithms

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Algorithms
2. Representation of Algorithms
3. Abstract Data Structures
    - Queues and Stacks
    - Graphs

Six handshakes, and how you would check

Universität Konstanz

MOTIVATING EXAMPLE

Milgram's 1967 experiment asked people in Nebraska to get a letter to a stranger in Boston by passing it only to someone they knew personally. The chains that arrived had a median of **five intermediaries**. Fifty years later the same question was answered directly over **1.59 billion people**, giving **3.57**.

5.2

Milgram, 1967: mean over the 64 letters that arrived

3.57

Facebook, 2016: 1.59×10^9 people

Milgram needed volunteers and postage. What *procedure* answers the same question from the network itself?

Milgram, *Psychology Today* (1967); Edunov, Diuk, Filiz, Bhagat & Burke, Facebook Research (2016)

Whose contacts, within two steps?

Universität Konstanz

MOTIVATING EXAMPLE

A case is confirmed on Monday. Public health needs everyone this person met, and everyone **those** people met, but not the whole population, and not next week. On the contact graph of a mid-sized city that is thousands of people, and it must be answered in **seconds**.

- “Everyone within two steps” is easy to say and useless until you specify *which person you look at next* and *how you know you are finished*.

What is the exact recipe, and how do you keep from going round in circles?

The network that would not fit

Universität Konstanz

MOTIVATING EXAMPLE

You want to store who is friends with whom among the **84 million** people in Germany. Written as a table of everyone against everyone, that is **7×10^{15} cells**, seven petabits at one bit each, more than most data centers hold. Written the other way round it fits on a laptop and answers the same questions.

7×10^{15}
cells, as a matrix

8×10^{9}
friendships, as adjacency lists

~$10^{6}\times$
smaller, same facts

Same network, same questions. Why does the *representation* decide whether the study can happen?

What is an algorithm?

Universität Konstanz

Definition

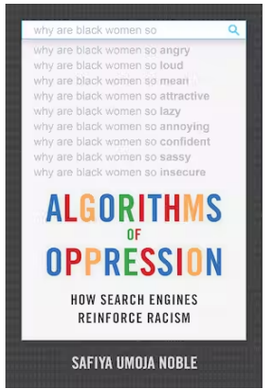
An algorithm is a self-contained step-by-step set of operations to solve a given problem. Algorithms perform calculation, data processing, and/or automated reasoning tasks.

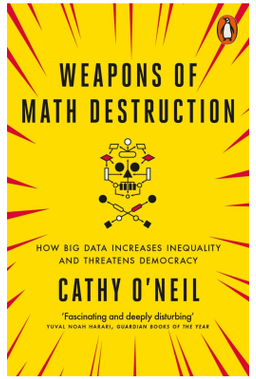
Other uses of the word algorithm

The word *algorithm* is commonly used nowadays with slightly different meanings, especially in the context of Artificial Intelligence:

- Algorithm as **method to fit a predictive model**. Can be as simple as least squares to fit linear regression and as complicated as distributed stochastic gradient descent to fit neural networks
- Algorithm as **the model after training**. This focuses on the predictive model as a program with a behavior that is not carefully designed, but learned from empirical data.

Related terms to these other meanings: algorithmic bias, algorithmic media, machine behavior, algorithm auditing, algorithmic decision-making…





al-Khwarizmi: the namesake of algorithms

Universität Konstanz

Muhammad ibn Musa abu Djafar al-Khwarizmi
(780 – 850, Khwarazm, Uzbekistan)



- Popularized algebra and published the first systematic way to solve linear and quadratic equations
- His book on how to calculate with Hindu-Arabic numerals was translated into Latin with the title *Algoritmi de Numero Indorum*
- *Algoritmi* later changed to *Algorithmus* as a mix with *arithmos* in Greek (number)

- It is clear that algorithms predate him (e.g. in Babylon, Egypt, India, and Greece)
- He widely popularized the idea of systematic treatment of data (numbers)
- He emphasized doing this in as few steps as possible - thus using a decimal system with pen and paper rather than Roman numerals with an abacus

Algorithms: python example

Universität Konstanz

Procedure K-SMALLEST-ELEMENT(M, n, k)
Inputs:
M: an array
n: the number of elements in M
k: which smallest element to find ($k = 1$ is the minimum)
Result: the k-th smallest element of M
For $i = 1$ to k:
    $smallest \leftarrow i$
    For $j = i + 1$ to n:
        If $M[j] < M[smallest]$
            then $smallest \leftarrow j$
    swap $M[i]$ with $M[smallest]$
return $M[k]$

Example: kth smallest element in a list

`Inputs:` list `M` of length `n`, index `k`

`Data: i, j, m, min_val`

`Result:` the `k`-th smallest element of `M`

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Algorithms: properties

Universität Konstanz

Specification:

- **input specification:** required input such that a given output can be produced (many or no inputs)
- **output specification:** result to be produced by the algorithm

Example: kth smallest element in a list

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Algorithms: properties

Universität Konstanz

Transferability:

- input should be parameterizable
- different instances of the problem must be allowed as inputs

The example allows lists of any possible length, not just one length

It also allows to find any kth smallest element, not just for one k

Example: kth smallest element in a list

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Algorithms: properties

Universität Konstanz

Description:

- **finiteness:** must terminate after a finite number of steps
- **definiteness:** each step must be clear, well-defined and precise (no ambiguity)

Execution and return, alternatives:

- **deterministic:** same output for same input(s)
- **stochastic:** a random component makes the output vary for the same input(s)

Example: kth smallest element in a list

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Algorithms: properties

Universität Konstanz

Time complexity:

- effort the given algorithm needs for completion
- is dependent on specific implementation
- measured by counting the number of key operations such as comparisons in the sorting algorithm

Memory complexity:

- required memory during execution
- measured by counting the maximum memory space required by the algorithm

Example: kth smallest element in a list

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Algorithms: properties

Universität Konstanz

Correctness:

- **partial correctness:** whenever the algorithm terminates, its output is correct
- **terminates:** for correct inputs, the algorithm must always terminate in a finite number of steps

Example: kth smallest element in a list

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Representation of Algorithms

1. Algorithms
2. **Representation of Algorithms**
3. Abstract Data Structures
    - Queues and Stacks
    - Graphs

Universität Konstanz

How to create a new program?

- Evaluate your requirements
- Define a structure for programming with a method of representation for its algorithm:
  - Pseudocode
  - Structograms
  - Flow diagrams
  - UML Unified Modeling Language
- Start programming

Universität Konstanz

How to create a new program?

- Evaluate your requirements
- Define a structure for programming with a method of representation for its algorithm:
  - **Pseudocode**
  - Structograms
  - **Flow diagrams**
  - UML Unified Modeling Language
- Start programming

Pseudocode

- representation of the steps in an algorithm as a combination of natural language and mathematical notation
- often uses structural conventions of a programming language
- not for machine interpretation but for illustration purposes among humans

Procedure NAME(*inputs*)
Inputs: input variables
Data: auxiliary variables
Result: output variables
preparation
While not the trivial case:
    work
bookkeeping

Good pseudocode

Procedure NAME(*inputs*)
Inputs: input variables
Data: auxiliary variables
Result: output variables
preparation
While not the trivial case:
    work
bookkeeping

Specify **input** and **output** if not clear from the context (problem)

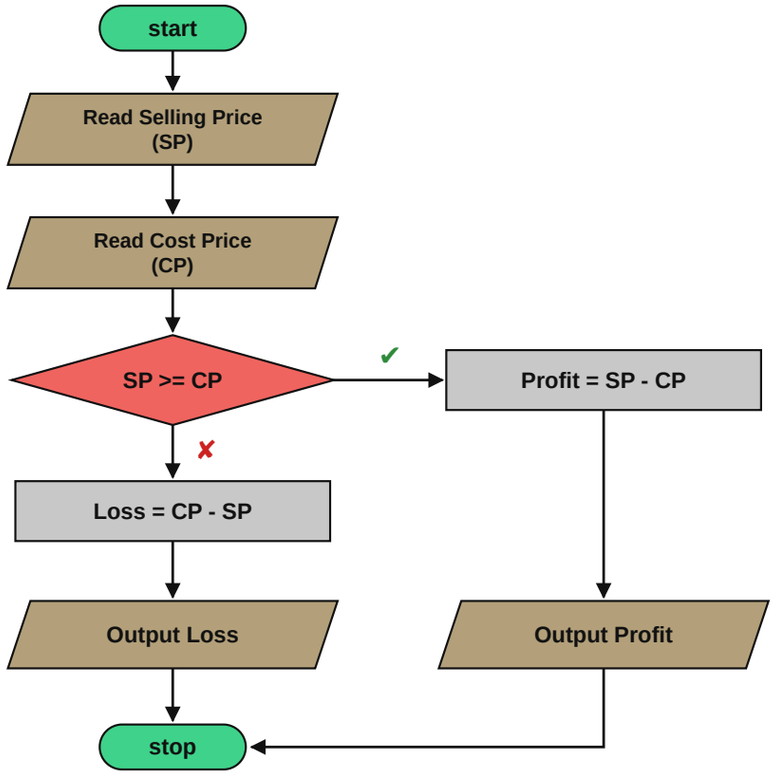
If you use auxiliary data structures, comment how to access and edit them

Make sure loops and conditional scopes are **unambiguous**

Bottom line: using your pseudocode, another human has to be able to follow its instructions to solve the problem with pen and paper (and later program it)

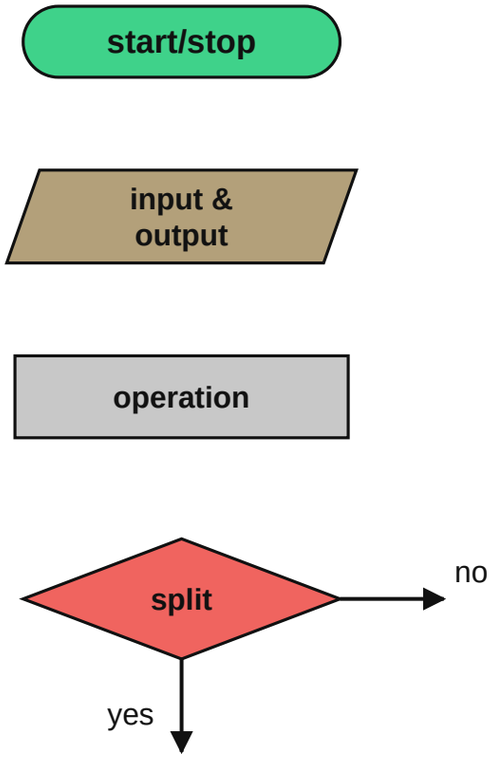
Flowcharts

- graphical representation of execution of an algorithm using symbols and transitions
- shows the steps as boxes, and their order by connecting the boxes with arrows

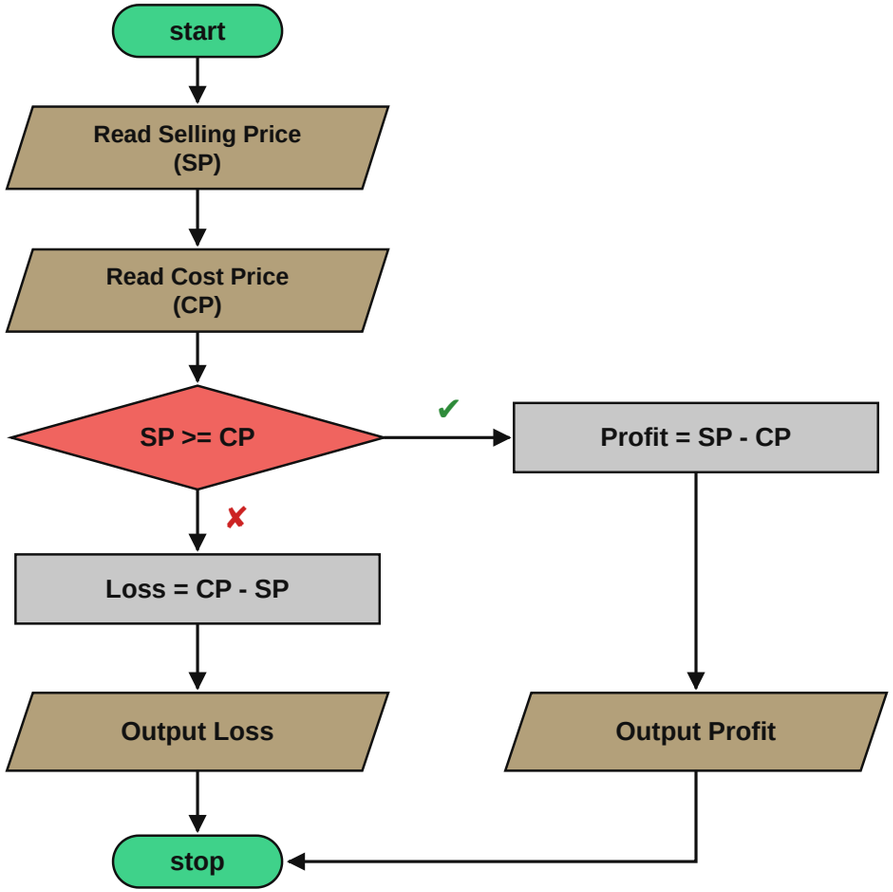


Flowcharts

- program start and end
  - beginning and ending of a program or sub-process
- data
  - process of inputting/entering and outputting/displaying data
- execute operations
  - set of operations that changes value, form, or location of data
- conditional split
  - conditional operation that determines the program path
  - commonly a yes/no question or true/false test



Flowchart example



- The program starts and reads selling price as SP
- Then reads cost price as CP
- The program tests if SP is larger than or equal CP
- If the condition is true, it takes the right path:
  - Calculate profit as SP-CP
  - Output profit
- if the condition is false, it takes the bottom path:
  - Calculate loss as CP - SP
  - Then output loss
- Program ends

Exercise: write pseudocode as a flowchart

Universität Konstanz

Example: kth smallest element in a list

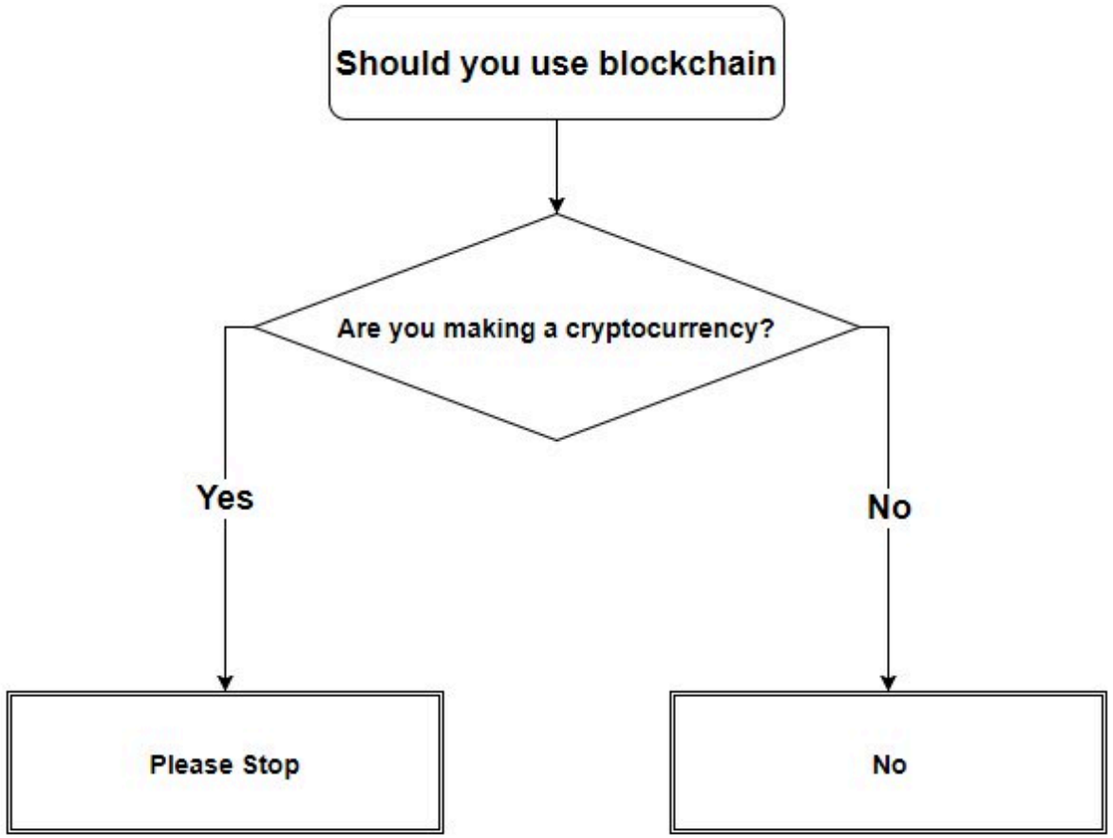
`Inputs:` list `M` of length `n`, index `k`

`Data: i, j, m, min_val`

`Result:` the `k`-th smallest element of `M`

```
 1 def min_k(M, k):
 2     for i in range(0, k):
 3         m = i
 4         min_val = M[i]
 5         for j in range(i+1, len(M)):
 6             if M[j] < min_val:
 7                 m = j
 8                 min_val = M[j]
 9         M[i], M[m] = M[m], M[i]
10     return M[k-1]
```

Another flowchart example



https://www.reddit.com/r/ProgrammerHumor/comments/7etmm1/a_diagram_to_help_you_decide_whether_blockchain/

Abstract Data Structures

Universität Konstanz

1. Algorithms
2. Representation of Algorithms
3. **Abstract Data Structures**
   - Queues and Stacks
   - Graphs

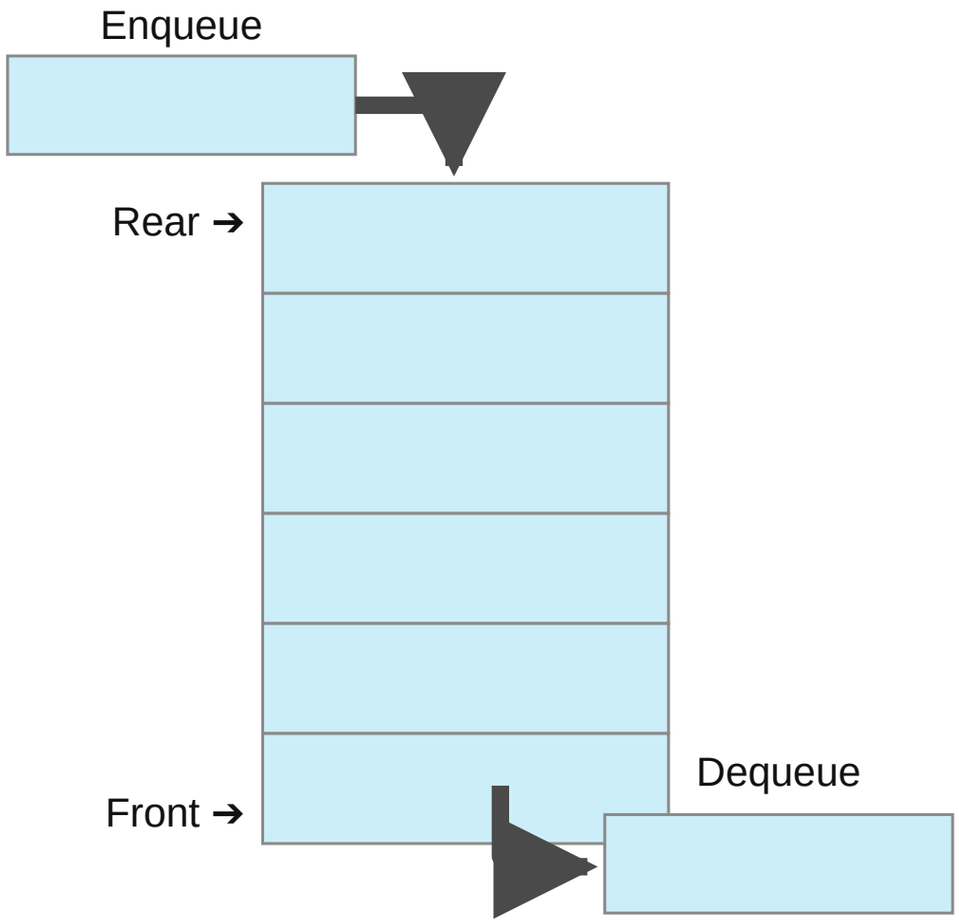
Abstract Data Structures

Universität Konstanz

- Characteristics
  - represent data in a way best suited to our (algorithmic) needs
    - arrangement, coding and link among data matter
  - allow for administration and access to data in an efficient way
  - exact specification is dependent on implementation and the use case
- We already saw a number of abstract data structures
  - arrays
  - linked lists
  - binary search trees
  - associative arrays (= dictionaries)
- Choice of data structure matters a lot for performance of specific algorithms
  - next: illustrative examples for each structure

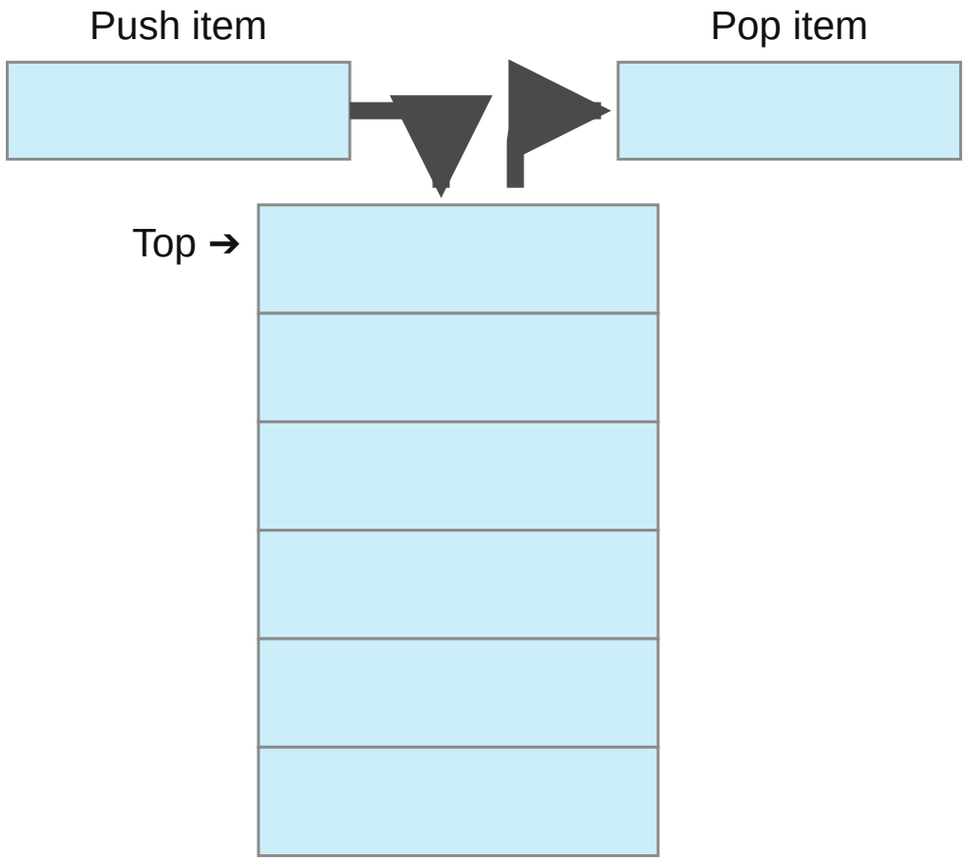
Queues

- Linear data structure following the **FIFO** principle: First In, First Out
- Two operations:
  - ENQUEUE(Q, v): add element v to the rear of the queue Q
  - DEQUEUE(Q): remove and return the element at the front of the queue Q
- Example applications:
  - processing requests in order
  - having several queues for priority



Stacks

- Linear data structure following the **LIFO** principle: Last In, First Out
- Two operations:
  - PUSH(S, v): add an element v to the top of the stack S
  - POP(S): remove and return the most recently added element, i.e. the top of the stack S
- Example applications:
  - Stack segment of the memory
    - management of layered execution of sub-routines
  - Recursion: more details in next session

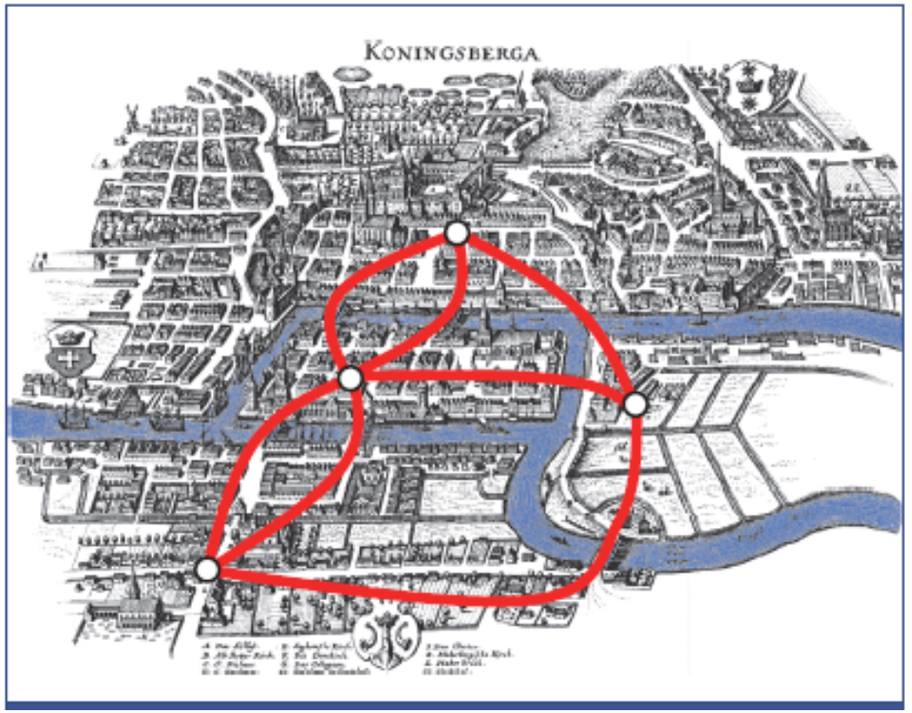


Graphs: motivation

Leonhard Euler: Seven bridges of Königsberg

Problem: Find a path that uses every bridge exactly once and ends in the same place.

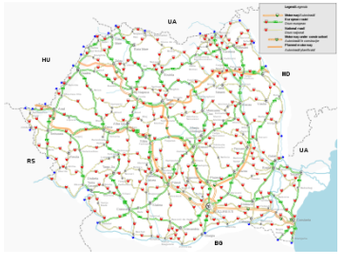
Example of a graph representation in a problem that motivated graph theory



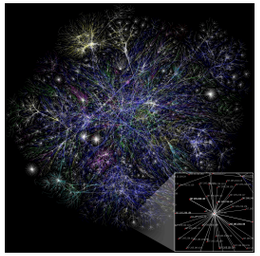
By Peter Berger: *Einführung in die Graphentheorie*

Graph examples

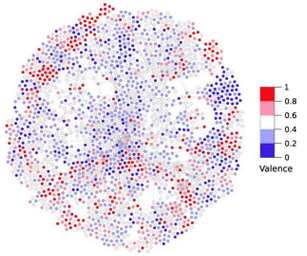
Many real-world structures can be represented as a graph. Examples are:



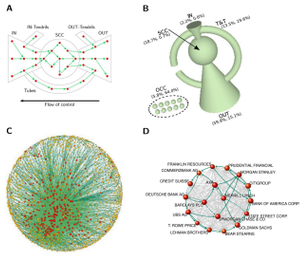
A road map



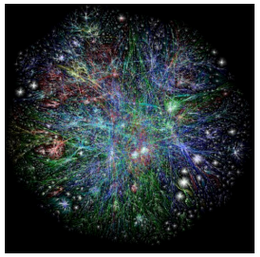
The Internet



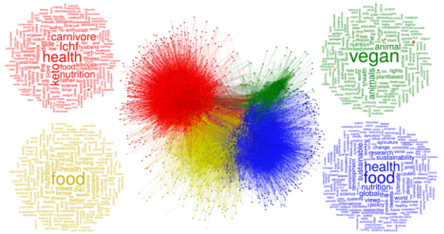
Word associations



Ownership between companies



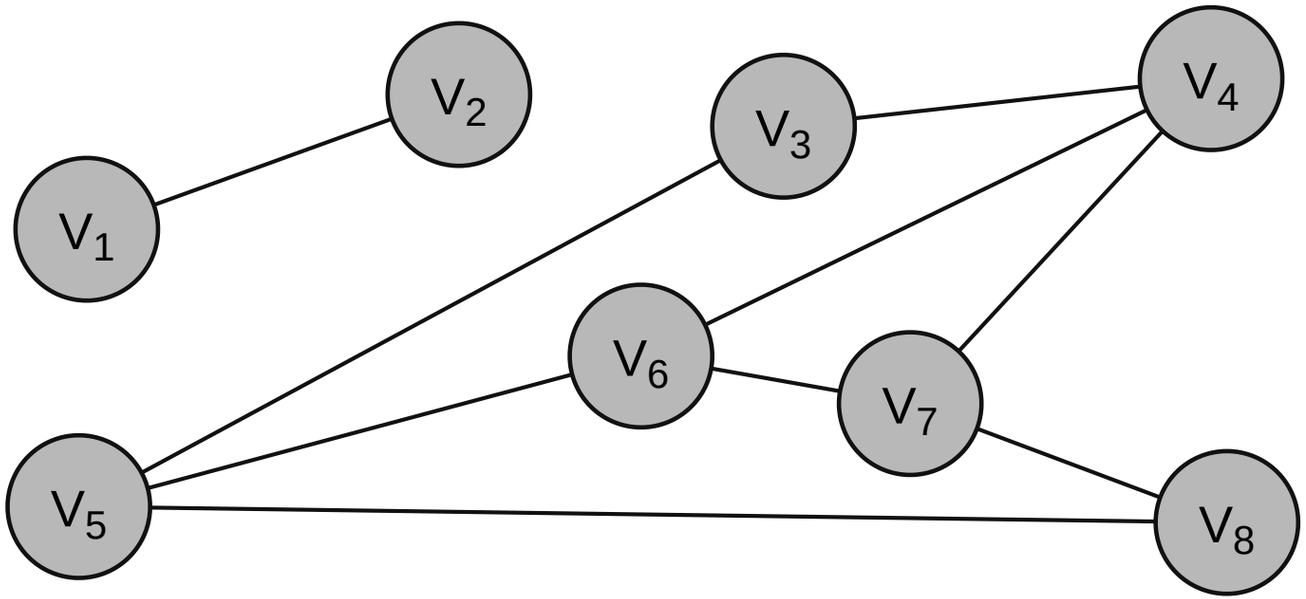
The Web



Social relations

Graphs

- consist of **vertices** and **edges**
- especially suitable to represent interdependencies between data points



Graphs

Universität Konstanz

- **vertices** represent any object
  - cities, physical states, atoms, steps in a process, persons
  - can be labeled or not
- **edges** represent the relationship between nodes
  - roads between cities, transitions between physical states, atomic bonds etc.
  - edges can have labels and variables, for example
    - physical distance (e.g. 2 Km in a map)
    - type of connection (e.g. friends or enemies)
    - weight of connection (e.g. strong versus weak)
  - can be **directed** or **undirected**

Graphs: Formalization

A graph G is a tuple $G = (V, E)$, where

- V is the set of all **vertices**
- E is the set of all **edges**

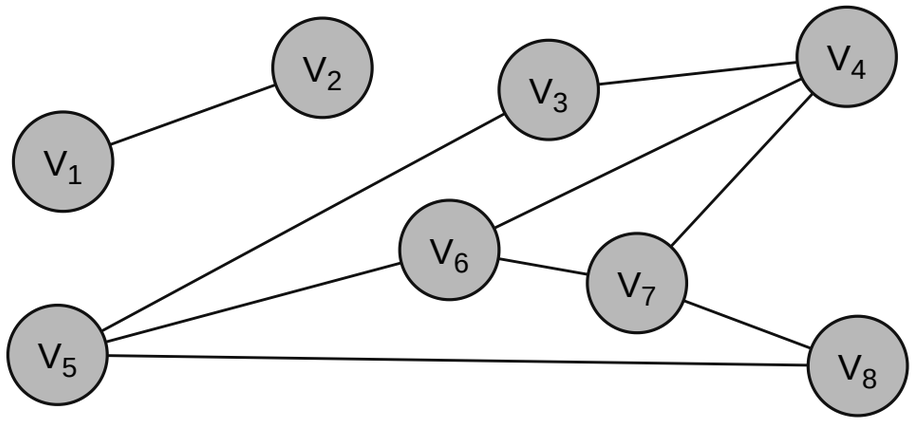
- For **directed** graphs, E consists of ordered pairs (v_1, v_2), i.e., the edge goes from v_1 to v_2
- For **undirected** graphs, E consists of unordered pairs $\{v_1, v_2\}$
- How do we represent graphs as an abstract data structure?

Adjacency Matrix

Universität Konstanz

Definition of Adjacency Matrix

If $G = (V, E)$ is a graph then two vertices $v, w \in V$ are **adjacent** if they are connected by an edge $\{v, w\} \in E$. The matrix $A(G)$ is called **adjacency matrix** if $a_{v,w} = 1$ if $\{v, w\} \in E$ and $a_{v,w} = 0$ otherwise.



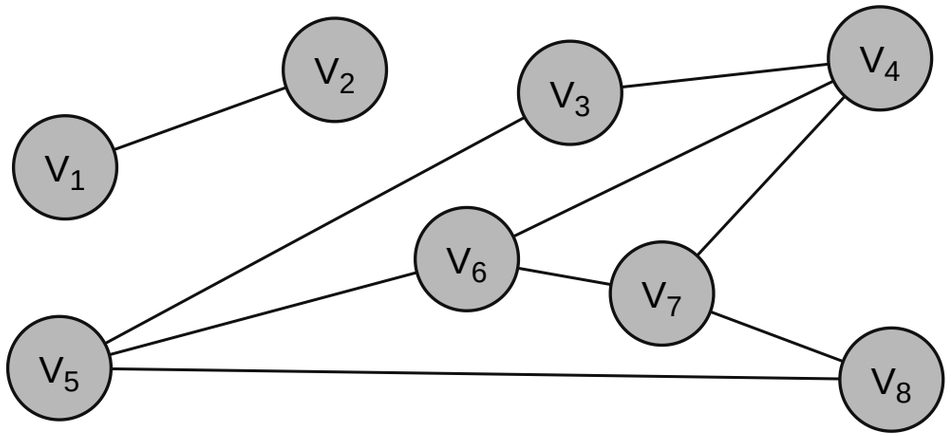
$$A(G) = \begin{array}{c|cccccccc} & v_1 & v_2 & v_3 & v_4 & v_5 & v_6 & v_7 & v_8 \\ \hline v_1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ v_2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ v_3 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ v_4 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ v_5 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ v_6 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ v_7 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ v_8 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{array}$$

Adjacency list

Universität Konstanz

Adjacency list: Alternative representation to adjacency matrix

- array of vertices in which all neighbors are stored as a linear list
- compressed representation of an adjacency matrix where all zeros have been eliminated. More memory efficient for sparse graphs.



v_1	$\rightarrow v_2$
v_2	$\rightarrow v_1$
v_3	$\rightarrow v_4 \rightarrow v_5$
v_4	$\rightarrow v_3 \rightarrow v_6 \rightarrow v_7$
v_5	$\rightarrow v_3 \rightarrow v_6 \rightarrow v_8$
v_6	$\rightarrow v_4 \rightarrow v_5 \rightarrow v_7$
v_7	$\rightarrow v_4 \rightarrow v_6 \rightarrow v_8$
v_8	$\rightarrow v_5 \rightarrow v_7$

Adjacency Matrix vs Adjacency List

Universität Konstanz

Comparing adjacency matrix and list representation

- example 1: does an edge exist between v_i and v_j?
  - adjacency matrix: direct access
  - adjacency list: iterate through all neighbors
- example 2: how many edges does a vertex v_i have?
  - adjacency matrix: iterate through all vertices
  - adjacency list: iterate only through all neighbors
- choice of representation clearly matters for access efficiency

Adjacency lists are more memory efficient for **sparse** networks (few edges per node)

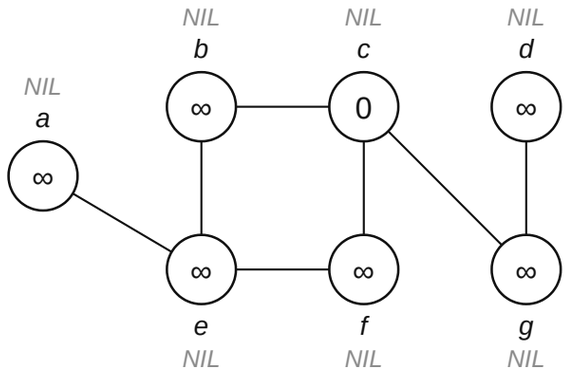
- If Facebook stored the friendship network data of 2.7 Billion users as an adjacency matrix and each entry in the matrix took just one bit, it would take 455 Petabytes (455,000 Terabytes) to store just the presence or absence of friendships between users.

Examples of Graph Problems

- Finding distances between nodes and shortest paths
- Finding subgraphs that are connected: i.e. there are paths between every two nodes in the subgraph
- Coloring nodes such that two neighbors do not have the same color
- Finding the nodes with the shortest average distance to all other nodes
- Ranking nodes by the number of shortest paths that go through them
- Finding a tree that connects all nodes with the lowest number of edges
- Finding a path connecting all nodes and using edges at most once
- Finding if two graphs can be mapped into each other (isomorphism)

Example: Finding distances

Let's consider this (undirected) graph $G = (V, E)$ with
$V = \{a, b, c, d, e, f, g\}$ and
$E = \{\{a, e\}, \{e, b\}, \{b, c\}, \{c, f\}, \{c, g\}, \{d, g\}, \{e, f\}\}$



Problem: Given a source node (aka starting node) s,

a. find all reachable nodes from it and

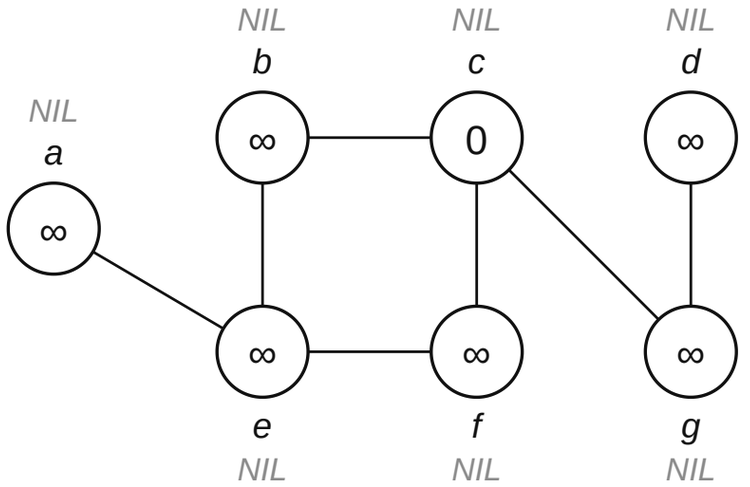
b. find a shortest path to each of them from s

$\rightarrow$ To do so use the **Breadth-First Search (BFS)** algorithm

Algorithm: Breadth-First Search

The first step is to initialize the graph with the INITIALIZE-BFS(G, s) procedure.

Procedure INITIALIZE-BFS(G, s)
Inputs:
G: an undirected graph
s: the starting node
Result: the graph is initialized for a breadth-first search starting at node s
For each vertex u in $G.V \setminus \{s\}$:
    color[u] ← WHITE
    d[u] ← ∞ # distance
    π[u] ← NIL # parent
color[s] ← GRAY
d[s] ← 0
π[s] ← NIL



Initialized with starting node $s = c$. The distance is inside each node, its name above or below it, and its parent in gray capitals (NIL for all nodes so far).

Algorithm: Breadth-First Search

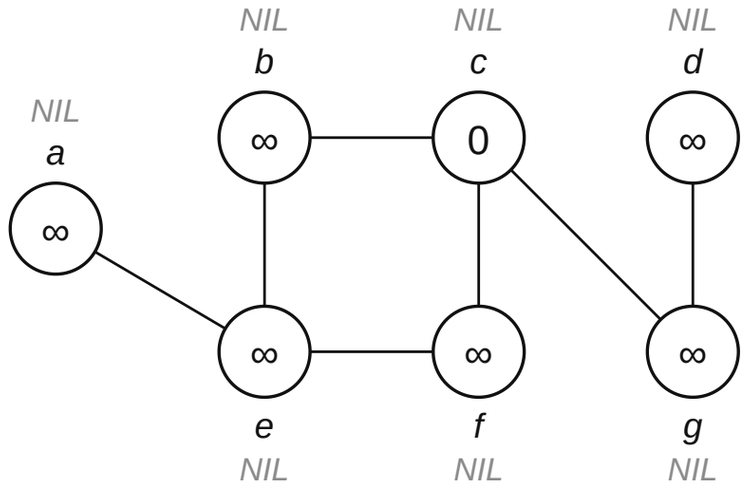
Universität Konstanz

Procedure BFS(G, s)
Inputs:
G: an undirected graph
s: the starting node
Result: every node reachable from s stores its distance d to s and its parent π
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s) # put s in the queue Q
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q) # take the next element from the queue Q
    For each vertex v in neighbors(u): # check all neighbors of u
        If color[v] = WHITE # only continue if v has not been seen yet
            then color[v] $\leftarrow$ GRAY # mark v as seen
                $d[v] \leftarrow d[u] + 1$ # one step further than u
                $\pi[v] \leftarrow u$ # u is the parent of v
                ENQUEUE(Q, v) # put v in the queue Q
    color[u] $\leftarrow$ BLACK # all neighbors of u have been processed

Algorithm: Breadth-First Search

Next initialize the Queue Q

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

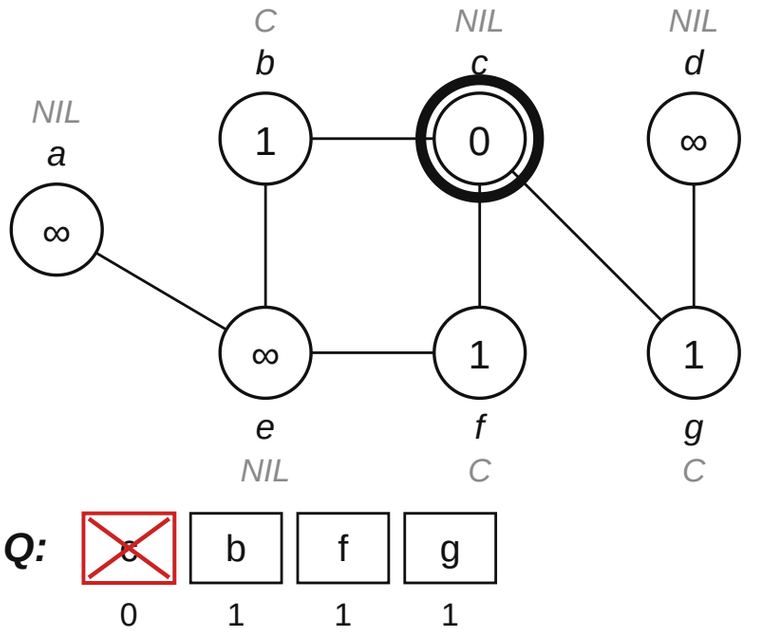


Q: c
0

Algorithm: Breadth-First Search

Iterate through the first while-loop for $u \leftarrow c$

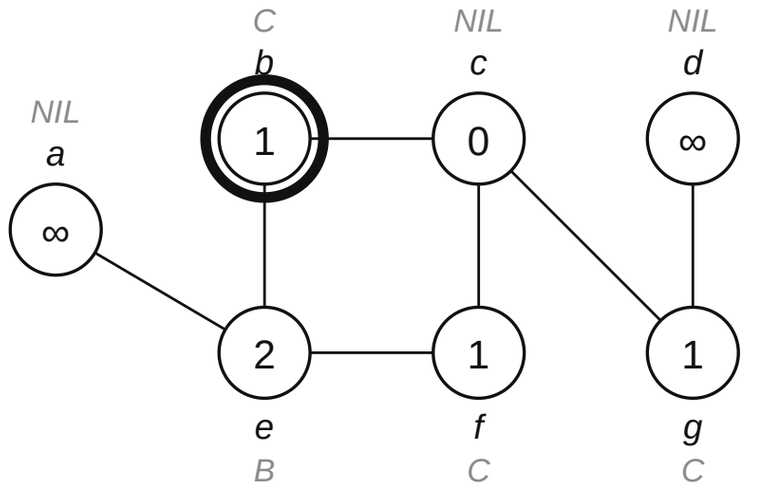
Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Algorithm: Breadth-First Search

Iterate through the next while-loop for $u \leftarrow b$ (b is next in the queue – FIFO)

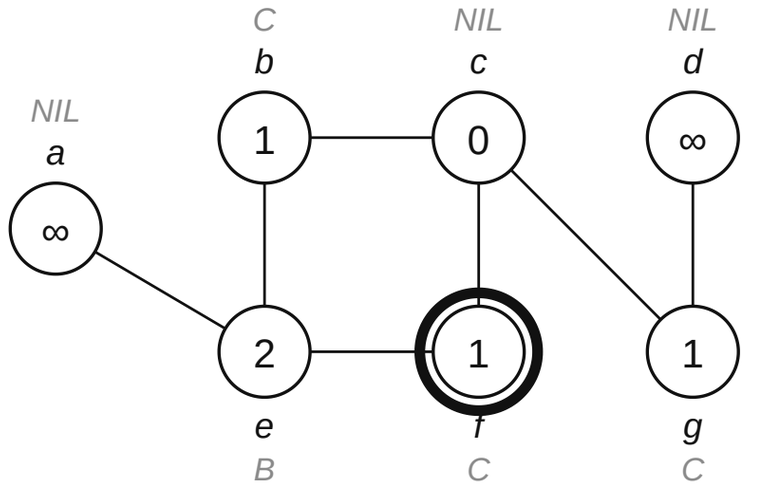
Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Algorithm: Breadth-First Search

Iterate through the next while-loop for $u \leftarrow f$

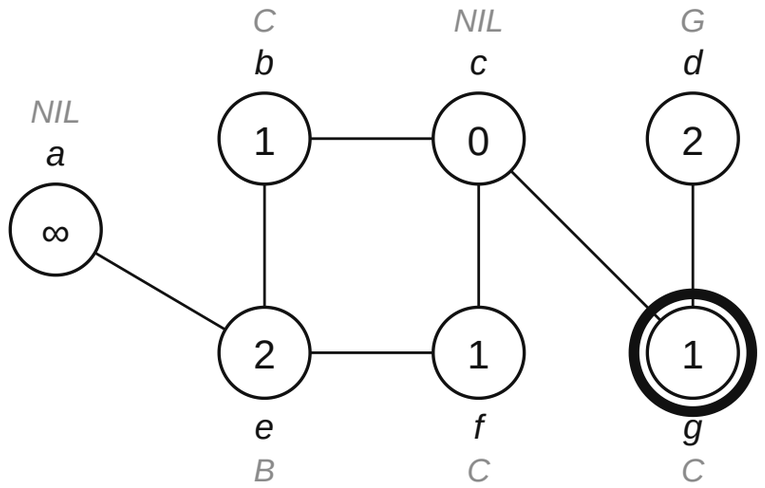
Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Algorithm: Breadth-First Search

Iterate through the next while-loop for $u \leftarrow g$

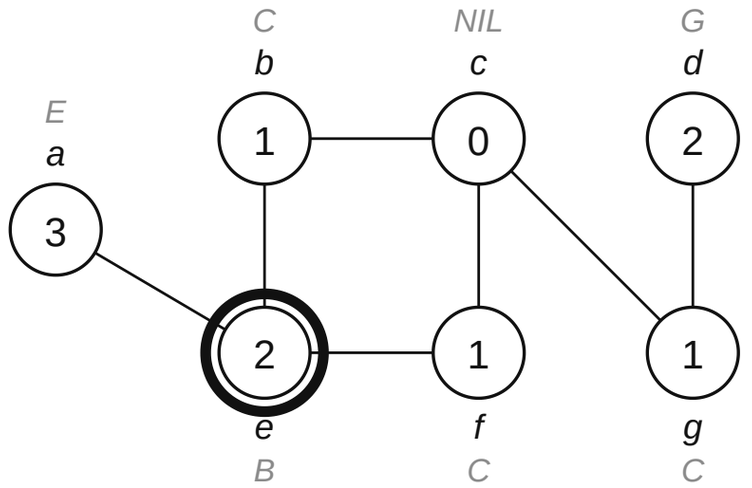
Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Algorithm: Breadth-First Search

Iterate through the next while-loop for $u \leftarrow e$

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK

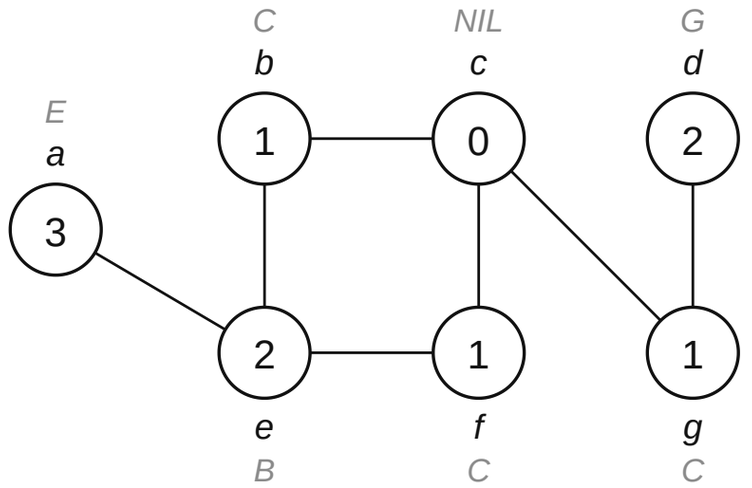


Algorithm: Breadth-First Search

Universität Konstanz

The while-loop for $u \leftarrow e$ finishes

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



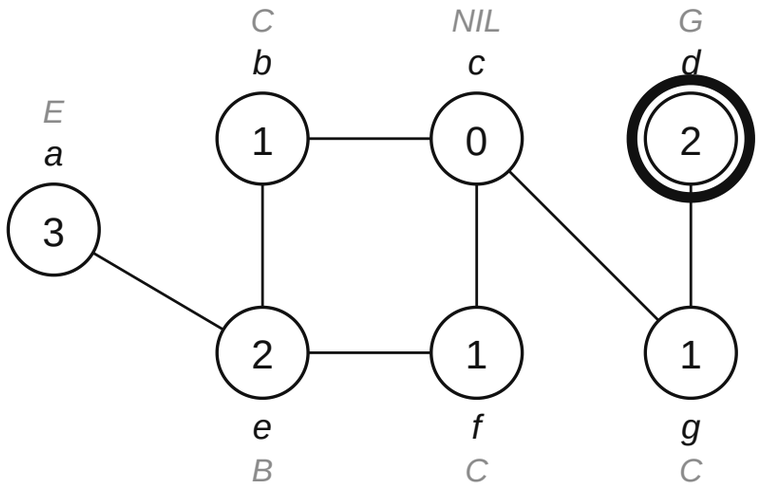
Q: d a

2 3

Algorithm: Breadth-First Search

Iterate through the next while-loop for $u \leftarrow d$

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Q: a

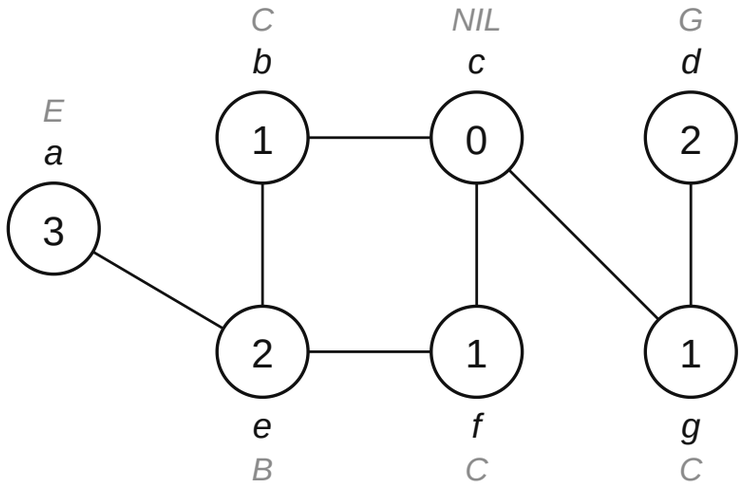
3

Algorithm: Breadth-First Search

Universität Konstanz

Iterate through the next while-loop for $u \leftarrow a$ **BFS Algorithm stops**

Procedure BFS(G, s)
Inputs: …
Result: …
INITIALIZE-BFS(G, s)
$Q \leftarrow \varnothing$
ENQUEUE(Q, s)
While $Q \neq \varnothing$:
    $u \leftarrow$ DEQUEUE(Q)
    For each vertex v in neighbors(u):
        If color[v] = WHITE
            then color[v] $\leftarrow$ GRAY
                $d[v] \leftarrow d[u] + 1$
                $\pi[v] \leftarrow u$
                ENQUEUE(Q, v)
    color[u] $\leftarrow$ BLACK



Q: ∅

Outtake – Social Science Applications

- Graphs in social science research
  - Social networks can be represented as graphs
    - classical work in sociology, then later also in political science, economics etc.
    - recent boom in research on this topic due to online social networks data
  - Many real-world structures can be represented as graphs
    - examples are road networks, flight routes, trade connections, treaty networks etc.
    - suite of tools available from mathematics, computer science, political methodology

Summary

Universität Konstanz

- An **algorithm** is a finite, well-defined sequence of steps
- It needs a clear **specification** and must be **finite**, **definite** and **correct**
- **Pseudocode** and **flowcharts** describe algorithms for humans
- **Queues** are first in, first out; **stacks** are last in, first out
- A **graph** $G = (V, E)$ is stored as an adjacency **matrix** or adjacency **lists**
- **Breadth-first search** uses a queue to find shortest distances

Next session: Recursion